# Text → Rigged, Animated Creature: Pipeline Plan

Oct 19, 2026 · @Paul

## Overview

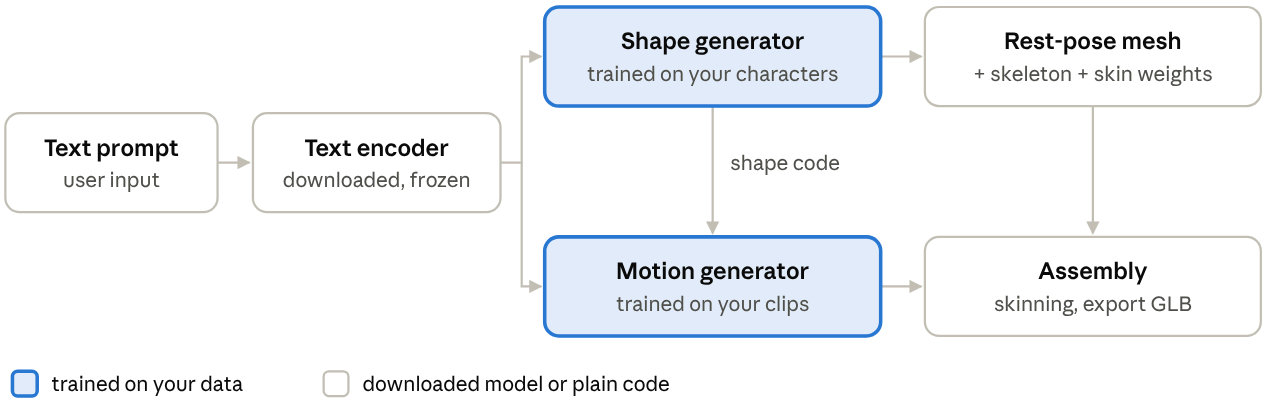
One text prompt goes in; a biped or quadruped comes out as a rest-pose mesh, a fitted skeleton, skin weights and an animation clip. Everything runs offline on a Galaxy S23 Ultra. The target budget is about 100–150 MB on disk and under 2 seconds per creature; both are estimates to confirm once the first model runs on the phone.

The system is four parts. Only parts 2 and 3 are trained on your data:

1. **Text encoder** (downloaded, frozen). Turns the prompt into a 512-number embedding.
2. **Shape generator** (trained by you). Turns the embedding plus the class (biped or quadruped) into a shape code of about 64–128 numbers. The code deforms that class's template mesh into the new creature. A learned joint regressor places the skeleton inside the new body, and the skin weights come with the template.
3. **Motion generator** (trained by you). Turns the embedding, class and shape code into per-frame joint rotations plus root movement, on the class's template skeleton.
4. **Assembly** (plain code, no learning). Applies the motion to the skinned mesh and exports glTF/GLB for the app or engine.

Shape and motion are separate models on purpose. Any learned body can then perform any learned motion, including combinations that never appear together in the dataset. The motion model sees the shape code so short legs take short steps and feet do not slide.

**One prompt drives two trained models that meet in assembly**




system architecture · 6 parts, 2 trained

Only the two highlighted models learn from your dataset; the shape code links them so motion fits the body.

## Dataset requirements

Each character needs a rest-pose mesh, its skeleton, skin weights, a class label and short captions. Each animation needs the clip on that character's skeleton plus a motion caption. Consistent conventions matter more than quantity: mixed scales, axes or rest poses are the most common cause of blurry, averaged results.

### What to provide per item

| Item | Required | Format | Notes |
|---|---|---|---|
| Rest-pose mesh | Yes | FBX or GLB (OBJ if separate skeleton file) | Triangles or quads, ideally under about 20k triangles. Watertight is not required. Textures are optional and not used for training. |
| Skeleton | Yes | Inside the FBX/GLB | Joint names, parent hierarchy, rest-pose transforms. |

| Item | Required | Format | Notes |
|---|---|---|---|
| Skin weights | Strongly recommended | Inside the FBX/GLB | Used to check registration quality and to validate the template weights. |
| Class label | Yes | `meta.json` | `biped` or `quadruped`. |
| Shape captions | Yes, 2–3 per character | `meta.json` | Body only, no motion: "stocky goblin with long arms, big pointed ears and short legs". Can be auto-generated, then reviewed. |
| Animation clips | Yes, for the motion model | FBX/GLB/BVH on the same skeleton | Frame rate recorded; root motion kept in the clip, not baked out. |
| Motion captions | Yes, 1–3 per clip | `meta.json` | Motion only, no body: "walks forward slowly, stops and looks left". |
| Loop flag | Recommended | `meta.json` | `true` for walk/run/idle cycles, `false` for one-shot actions. |

## Conventions to apply to every file

- **Units:** meters, real-world size.
- **Axes:** one up-axis and one forward-axis for the whole dataset (for example Y-up, facing +Z), feet on the ground plane at height 0.
- **Rest pose:** bipeds in the same pose (T-pose or A-pose, pick one); quadrupeds standing neutrally with legs straight down.
- **Joint names:** a `joint_map.json` per source rig that maps its joint names to the class template names. Joints the template does not have are listed as unmapped, not deleted.

## Folder layout

```
dataset/
  joint_maps/
    mixamo_biped.json
    studioX_quadruped.json
  characters/
    goblin_001/
      model.fbx          # rest-pose mesh + skeleton + skin weights
      meta.json          # class, shape captions, joint_map name
      anims/
        walk_01.fbx
        attack_02.fbx
        anims.json       # per clip: fps, loop flag, motion captions
    wolf_014/
      ...
```

Example `meta.json`:

```
{
  "id": "goblin_001",
  "class": "biped",
  "joint_map": "mixamo_biped",
  "shape_captions": [
    "small stocky goblin with long arms and big pointed ears",
    "short green creature, hunched, oversized hands"
  ]
}
```

## How much data

These are rough planning targets, not hard limits; the first training runs will show where the real limit sits.

| Part | Minimum per class | Good per class |
|---|---|---|
| Characters (shape model) | about 300 | 1,000 or more |
| Animation clips, generative motion | about 500 | 2,000 or more |
| Animation clips, clip selection + adaptation | 20–50 clip types | 100 or more |

Hold out about 10% of **characters** (not clips) for validation, so the model is tested on bodies it has never seen.

## What to download

You only download a text encoder, a captioning model for data prep, and tools. The shape and motion generators are small and trained from scratch on your data, because no pretrained model knows your two templates.

### Models

| Component | Download | Used for | Runs on | License |
|---|---|---|---|---|
| Text encoder (commercial-safe) | OpenAI CLIP ViT-B/32, text tower only | Prompt → embedding, in the app | Phone | MIT |
| Text encoder (research alternative) | MobileCLIP2-S0, text tower (about 63M parameters) | Same, slightly stronger | Phone | Apple research license (apple-amlr); redistribution restricted, so not for a shipped app |
| Captioning model | Qwen2.5-VL-7B-Instruct | Auto-captions for rendered characters and animation frames | Your PC GPU (data prep only) | Apache 2.0 |

### Tools and libraries

| Tool | Used for | Notes |
|---|---|---|
| Blender + `bpy` (Blender as a Python module) | Import FBX/GLB, normalize, render views for captions, export GLB | `bpy` on PyPI only works with the Python version its Blender release targets; check before installing. |
| PyTorch | All training | GPU build. |

| Tool | Used for | Notes |
|---|---|---|
| trimesh, libigl (Python), Open3D | Mesh loading, sampling, non-rigid registration | |
| Robust Skin Weights Transfer (Epic Games sample code) | Transfer template skin weights to registered meshes; check registration quality | A free Blender addon version also exists. |
| ONNX + ONNX Runtime with QNN Execution Provider | Export and run models on the Snapdragon NPU | LiteRT (TensorFlow Lite) is the alternative runtime. |

## Optional extra motion data

Only useful if your own clip count is low. Use them for pre-training the motion model, then fine-tune on your clips. Check each license before any commercial use.

| Dataset | Content | Status |
|---|---|---|
| AniMo4D | 78,149 animal motion sequences, 185,435 text descriptions, 114 species | Published (CVPR 2025) |
| Truebones Zoo + captions | Animal motion for 70+ species, text descriptions added by the paper's authors | Published |
| QuadFM | 11,784 quadruped clips with 35,352 descriptions | Announced; release pending |
| HumanML3D | Human motion with text, for bipeds | Built on AMASS, research and non-commercial terms |

These datasets use their own skeletons, so their clips must be retargeted to your templates first (stage 8 below).

# Pipeline stages

Stages 1–6 turn your files into training data; the hardest is stage 3, registration, because every later model depends on its quality. Stages 7–10 train and evaluate the two generators.

## Data preparation

1. **Ingest and normalize** (Blender script). Import each character, apply transforms, convert to meters and the shared axes, put feet on the ground, check the rest pose. Save mesh, skeleton, weights and clips as `.npz`.
   - Output: one clean `.npz` per character and per clip, plus a report of files that failed checks.
2. **Build one template per class.** Pick a representative biped and quadruped, clean the topology to about 5–8k vertices, and build the superset skeleton: every joint any character in that class has (tails, ears, wings), each with an "optional" flag. Paint good skin weights once, by hand.
   - Output: `template_biped.glb`, `template_quadruped.glb`.
3. **Register every character to its template.** First pose the template skeleton onto the character's skeleton, using the joint map, and deform the template with its own skin weights. This gives a close starting fit. Then run non-rigid ICP to fit the surface exactly. Score each fit and send poor ones for manual review.
   - Output: every character as the same 5–8k vertices in the same order.
4. **Skin weights and joint regressor.** Every registered character now shares the template's weights. Check them against the original weights with the skin-weight-transfer tool. Then learn a joint regressor: each joint position as a fixed weighted average of nearby vertices, fitted across all characters.
   - Output: `joint_regressor.npy`, so any generated mesh gets its skeleton automatically.
5. **Build the shape space.** Run PCA on the registered vertex offsets from the template to get a shape basis of 64–128 components. Add a small neural decoder later only if PCA loses detail you care about.
   - Output: `shape_basis.npy` and one shape code per character.
6. **Caption.** Render 4 views of each character and caption them with Qwen2.5-VL for shape. Render frames of each clip and caption them for motion. Review a random 5–10% by hand and fix the prompt where captions are wrong.

- Output: captions written into `meta.json` and `anims.json`.

### Training

7. **Train the shape generator.** Input: text embedding and class. Output: shape code. Use a small conditional diffusion or flow model over the code so one prompt can give varied creatures. Drop the text 10% of the time during training to enable classifier-free guidance.
8. **Retarget and prepare motion.** Map every clip onto its class template skeleton. Store per frame: joint rotations (6D form), root velocity and height, and foot-contact labels. Resample to 30 fps and cut into windows of up to about 6 seconds.
9. **Train the motion generator.** Train a motion tokenizer (VQ-VAE) on the clips, then a masked transformer that predicts motion tokens from the text embedding, class and shape code. Add a foot-contact loss to prevent sliding. If clips are few or mostly fixed types, use clip selection plus adaptation instead.
10. **Evaluate on held-out characters.**
    - Shape: surface distance to the real mesh, and whether the text encoder ranks the right caption first for the generated shape.
    - Skeleton: average joint position error in centimeters.
    - Motion: foot sliding, ground penetration, and a side-by-side review of generated vs. real clips.

## Prompt and motion understanding

The prompt is split into a body part and a motion part, each mapped into its own embedding space; motion is then understood as a sequence of learned phrases checked against kinematic rules. This layer replaces the plain "text embedding in, frames out" motion generator from stage 9 and needs less data to work well.

### 1. Prompt parsing

One prompt usually mixes body and motion: "a fat goblin limps forward and waves". A small parser splits it before anything else runs.

| Part | Example | Goes to |
|---|---|---|
| Class cue | goblin → biped; wolf → quadruped | class selector |

| Part | Example | Goes to |
|---|---|---|
| Body description | fat goblin | shape generator |
| Motion phrases, in order | limps forward; waves | motion phrase grammar |
| Style modifiers | slowly, angrily, tired | motion style vector |

On the phone this parser can be a tiny classifier head over the CLIP token embeddings, or a rule-based keyword matcher for a first version. Both run in milliseconds. During training the split comes free: shape captions and motion captions are already separate in `meta.json` and `anims.json`.

## 2. Joint text–motion embedding

Train a motion encoder so that a clip and its caption land close together in the same 512-number space as the frozen CLIP text embedding (contrastive training, as in TMR and MotionCLIP). This gives three things:

- **Retrieval.** Any prompt can be matched to the nearest training clips. This is the clip-selection path, and it works from day one with a few hundred clips.
- **A quality metric.** Generated motion is encoded and compared to the prompt; a low similarity flags a bad generation before the user sees it.
- **A conditioning signal.** The motion generator reads this embedding instead of raw text, so unseen phrasings ("saunters", "strolls") still land near "walks slowly".

Train it on your clips first; the extra animal datasets listed above can pre-train it if clip counts are low.

## 3. Motion phrase grammar

Motion is modelled as a sequence of phrases, not raw frames. Each phrase is a short learned primitive on the class template (an idle loop, one walk cycle, a jump, an attack swing) plus a transition rule to the next phrase.

```
Motion     := Phrase (Transition Phrase)*
Phrase     := Loop | Action
Loop       := idle | walk | trot | run | fly      # repeats until a
transition
Action     := jump | attack | sit | look_around   # plays once
Transition := blend(frames) | contact_aligned     # chosen by foot phase
```

The parsed motion phrases pick the sequence; the style vector (speed, energy, mood) modulates each phrase through a small conditioning network. The phrase set differs per class: a quadruped has gaits (walk, trot, gallop) that a biped does not.

This structure gives loopable, blendable output that an engine can use directly, and it degrades gracefully: an unknown verb falls back to the nearest phrase in the joint embedding space rather than producing garbage frames.

## 4. Kinematic rule set

Every generated frame is checked, and if needed corrected, against explicit rules for its template. These are written once per class, not learned.

| Rule | Biped | Quadruped |
|---|---|---|
| Joint limits | knee and elbow hinge, one axis, 0–150° | stifle and hock hinge; spine flex within ±30° |
| Bone length | fixed to the generated skeleton | same |
| Symmetry | left/right joint limits mirrored | same |
| Foot contact | a foot in contact does not slide or sink | same, per leg, following gait phase |
| Root motion | root velocity matches foot step length | same, scaled by leg length from the shape code |
| Balance (optional) | centre of mass stays over the support polygon in loops | same |

The rules are used in three places: as losses while training the motion model, as a post-process projection on the phone (cheap inverse kinematics on the feet), and as an automatic test on every dataset clip during preparation, which also catches retargeting errors.

## What this changes in the plan

- Stage 6 (captioning) also tags each clip with its phrase labels and style words.
- Stage 8 (motion prep) segments clips into phrases at foot-contact boundaries.

- Stage 9 (motion model) becomes: train the joint embedding, then the phrase primitives, then a small sequencer that maps parsed phrases plus style to a phrase sequence. Free-form frame generation is a later upgrade, not the first version.
- Phone budget: the parser and joint embedding add under 10 MB.

## On-device deployment (Galaxy S23 Ultra)

The whole system should fit in about 90–120 MB, with the text encoder taking more than half. All sizes below are estimates until measured on the phone.

| Part | Format on phone | Estimated size | Where it runs |
|---|---|---|---|
| Text encoder, int8 | ONNX or LiteRT | about 60–65 MB | NPU or CPU |
| Shape generator | ONNX | 2–5 MB | NPU or GPU |
| Shape basis + templates + skin weights | binary arrays | about 8–10 MB | app code |
| Joint regressor | sparse binary array | under 1 MB | app code |
| Motion tokenizer decoder + transformer | ONNX | 10–40 MB | NPU or GPU |

For scale: published int8 exports of the MobileCLIP2-S0 text encoder come to 61.3 MB and 65 MB. The CLIP ViT-B/32 text tower is a similar size.

### Export rules

- **Export one denoising step, not the loop.** The QNN Execution Provider supports only a subset of ONNX operators, and Loop and If are among those not supported. Keep diffusion steps and token sampling as a loop in app code.
- **Quantize to int8** with ONNX Runtime's quantization tools, then compare outputs against the PyTorch model on the validation set.
- **Test on the actual phone early.** ONNX Runtime documents QNN testing on other Snapdragon chips; the S23 Ultra's Snapdragon 8 Gen 2 should work but needs checking. If an operator fails on the NPU, ONNX Runtime falls back to CPU for that

part.

### In the app

1. Tokenize the prompt and run the text encoder.
2. Run the shape generator loop → shape code → mesh = template + basis × code.
3. Joint regressor × mesh → skeleton. Skin weights come from the template.
4. Run the motion generator loop → motion tokens → joint rotations per frame.
5. Export glTF/GLB, or hand mesh, bones and clip straight to the engine. Native Android uses ONNX Runtime's Android package; Unity can load the ONNX models through its own inference engine.

## Checklist

Start with 10–20 characters per class end to end before preparing the whole dataset, so problems in conventions show up early.

**Your dataset prep**

- [ ] Pick one up-axis, forward-axis, unit (meters) and rest pose per class, and write them down
- [ ] Export every character as FBX or GLB with mesh, skeleton and skin weights in one file
- [ ] Write a `joint_map.json` for each source rig
- [ ] Add `meta.json` with class and 2–3 shape captions (or leave captions empty for auto-captioning)
- [ ] Put each character's clips in `anims/` with fps, loop flag and motion captions in `anims.json`
- [ ] Choose the template character for each class
- [ ] Decide commercial or research use, which picks the text encoder

**Downloads**

- [ ] Blender and a matching `bpy` Python environment
- [ ] PyTorch with GPU support, trimesh, libigl, Open3D
- [ ] CLIP ViT-B/32 (or MobileCLIP2-S0 for research)
- [ ] Qwen2.5-VL-7B-Instruct, if captions need generating
- [ ] ONNX Runtime with the QNN Execution Provider for the phone build

**Open questions**

- Do all rigs within a class share joint names, or are there several source rigs?
- Roughly how many characters and clips per class?
- Native Android app, or Unity/Unreal?
- Which GPU is available for training?

## Sources


- MobileCLIP2-S0 model card (parameters, license)
- MobileCLIP2-S0 int8 ONNX export (text encoder size)
- Sift: MobileCLIP2-S0 on Android with LiteRT (size, license note)
- Qwen2.5-VL-7B-Instruct (Apache 2.0)
- ONNX Runtime QNN Execution Provider
- Robust Skin Weights Transfer via Weight Inpainting, sample code
- Robust Weight Transfer, Blender addon
- AniMo and AniMo4D (CVPR 2025)
- How to Move Your Dragon (Truebones Zoo captions)
- QuadFM dataset paper